

기본적인 영상 처리 기법

1) 영상의 밝기 조절

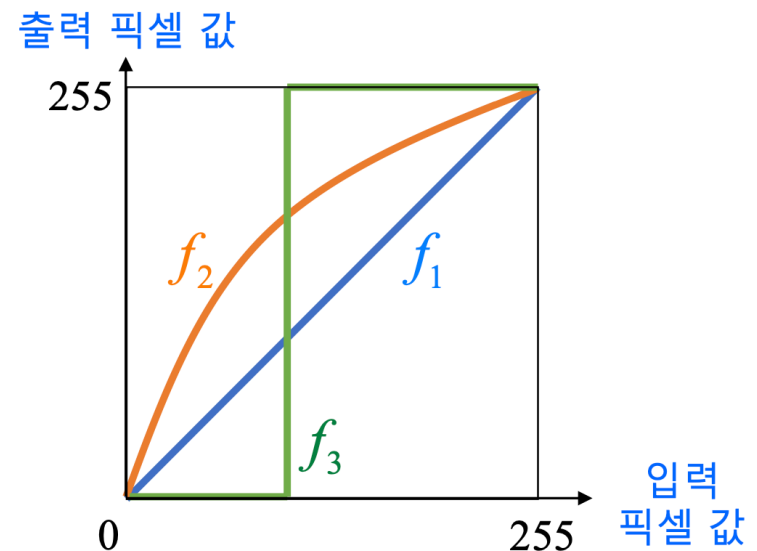
영상의 화소 처리 기법

■ 화소 처리(Point processing)

- 입력 영상의 특정 좌표 픽셀 값을 변경하여 출력 영상의 해당 좌표 픽셀 값으로 설정하는 연산

$$\text{dst}(x, y) = f(\text{src}(x, y))$$

변환 함수
(transfer function)



- 결과 영상의 픽셀 값이 정해진 범위(e.g. 그레이스케일)에 있어야 함
- 반전, 밝기 조절, 명암비 조절 등

영상의 밝기 조절

■ 밝기 조절이란?

- 영상을 전체적으로 더욱 밝거나 어둡게 만드는 연산



원본 영상

밝기 -50 조절



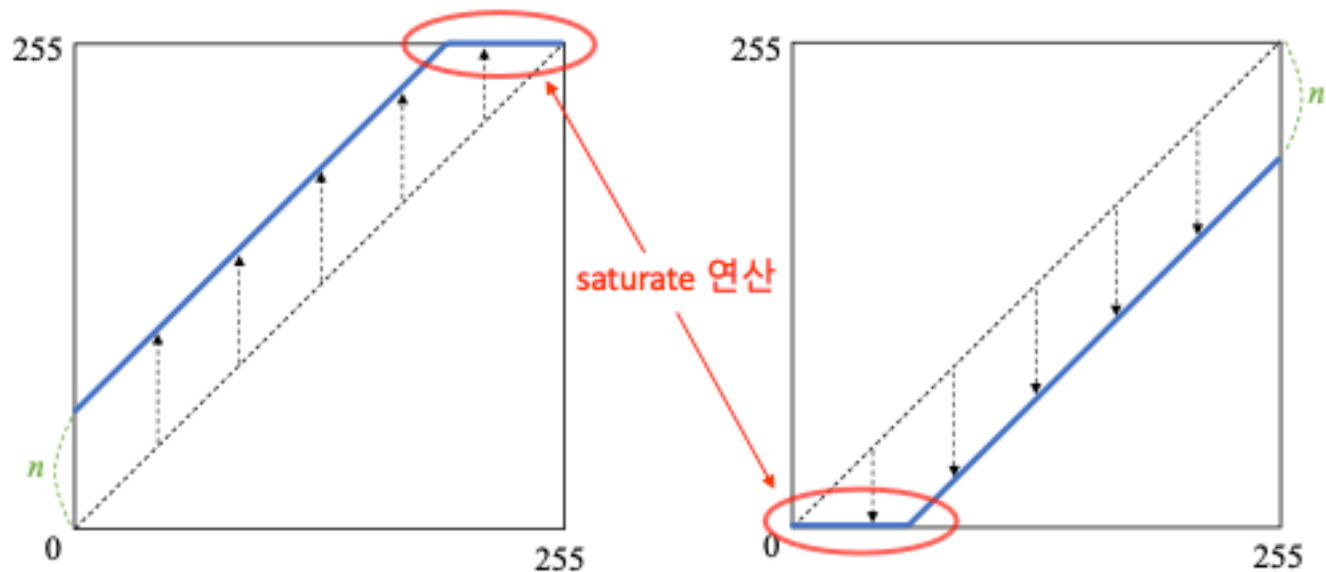
밝기 +50 조절



영상의 밝기 조절

■ 밝기 조절 수식

$$\text{dst}(x, y) = \text{saturate}(\text{src}(x, y) + n)$$



영상의 밝기 조절

■ 영상의 밝기 조절을 위한 영상의 덧셈 연산

```
cv2.add(src1, src2, dst=None, mask=None, dtype=None) -> dst
```

- src1: (입력) 첫 번째 영상 또는 스칼라
- src2: (입력) 두 번째 영상 또는 스칼라
- dst: (출력) 덧셈 연산의 결과 영상
- mask: 마스크 영상
- dtype: 출력 영상(dst)의 타입. (e.g.) cv2.CV_8U, cv2.CV_32F 등
- 참고사항
 - 스칼라(Scalar)는 실수 값 하나 또는 실수 값 네 개로 구성된 튜플
 - dst를 함수 인자로 전달하려면 dst의 크기가 src1, src2와 같아야 하며, 타입이 적절해야 함

영상의 밝기 조절

- 그레이스케일 영상의 밝기 100만큼 증가시키기

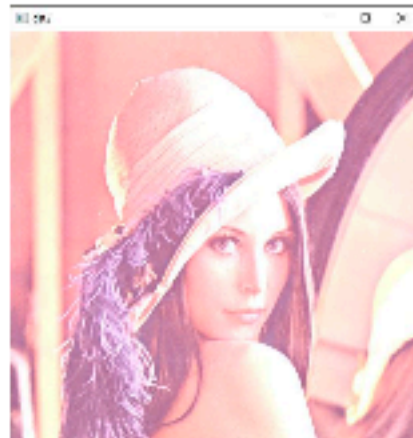
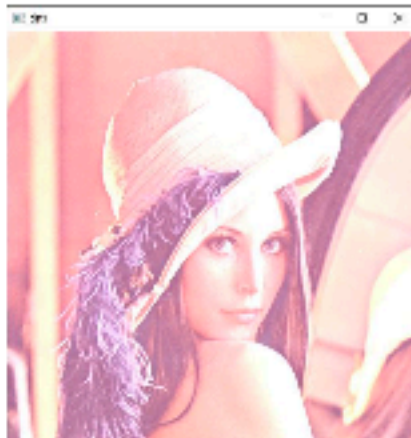
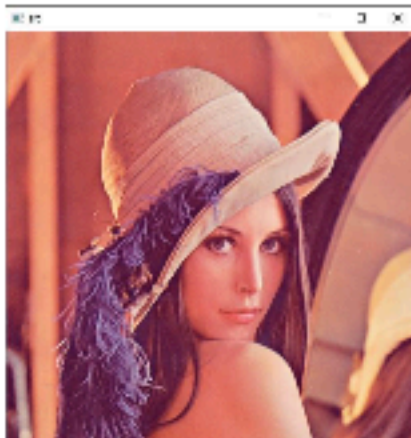
```
src = cv2.imread('lenna.bmp', cv2.IMREAD_GRAYSCALE)  
  
dst1 = cv2.add(src, 100)  
  
dst2 = src + 100  
#dst2 = np.clip(src + 100., 0, 255).astype(np.uint8)
```



영상의 밝기 조절

■ 컬러 영상의 밝기 100만큼 증가시키기

```
src = cv2.imread('lenna.bmp')  
dst1 = cv2.add(src, (100, 100, 100, 0))  
dst2 = np.clip(src + 100., 0, 255).astype(np.uint8)
```



기본적인 영상 처리 기법

2) 영상의 산술 및 논리 연산

영상의 산술 연산

▪ 덧셈 연산

$$\text{dst}(x, y) = \text{saturate}(\text{src1}(x, y) + \text{src2}(x, y))$$

- 두 영상의 같은 위치에 존재하는 픽셀 값을 더하여 결과 영상의 픽셀 값으로 설정
- 덧셈 결과가 255보다 크면 픽셀 값을 255로 설정 (포화 연산)



+



=



영상의 산술 연산

▪ 덧셈 연산

```
cv2.add(src1, src2, dst=None, mask=None, dtype=None) -> dst
```

- src1: (입력) 첫 번째 영상 또는 스칼라
- src2: (입력) 두 번째 영상 또는 스칼라
- dst: (출력) 덧셈 연산의 결과 영상
- mask: 마스크 영상
- dtype: 출력 영상(dst)의 타입. (e.g.) cv2.CV_8U, cv2.CV_32F 등
- 참고사항
 - 스칼라(Scalar)는 실수 값 하나 또는 실수 값 네 개로 구성된 튜플
 - dst를 함수 인자로 전달하려면 dst의 크기가 src1, src2와 같아야 하며, 타입이 적절해야 함

영상의 산술 연산

■ 가중치 합(weighted sum)

$$\text{dst}(x, y) = \text{saturate}(\alpha \cdot \text{src1}(x, y) + \beta \cdot \text{src2}(x, y))$$

- 두 영상의 같은 위치에 존재하는 픽셀 값에 대하여 가중합을 계산하여 결과 영상의 픽셀 값으로 설정
- 보통 $\alpha + \beta = 1$ 이 되도록 설정 → 두 입력 영상의 평균 밝기를 유지

■ 평균 연산(average)

- 가중치를 $\alpha = \beta = 0.5$ 로 설정한 가중치 합

$$\text{dst}(x, y) = \frac{1}{2}(\text{src1}(x, y) + \text{src2}(x, y))$$

영상의 산술 연산

■ 가중치 합(weighted sum)

$$\frac{1}{2} \left(\text{Image 1} + \text{Image 2} \right) = \text{Result}$$



$\alpha = 1, \beta = 0$



$\alpha = 0.75, \beta = 0.25$



$\alpha = 0.5, \beta = 0.5$



$\alpha = 0.25, \beta = 0.75$



$\alpha = 0, \beta = 1$

원본이미지 필요

■ 가중치 합(weighted sum)

```
cv2.addWeighted(src1, alpha, src2, beta, gamma, dst=None, dtype=None) -> dst
```

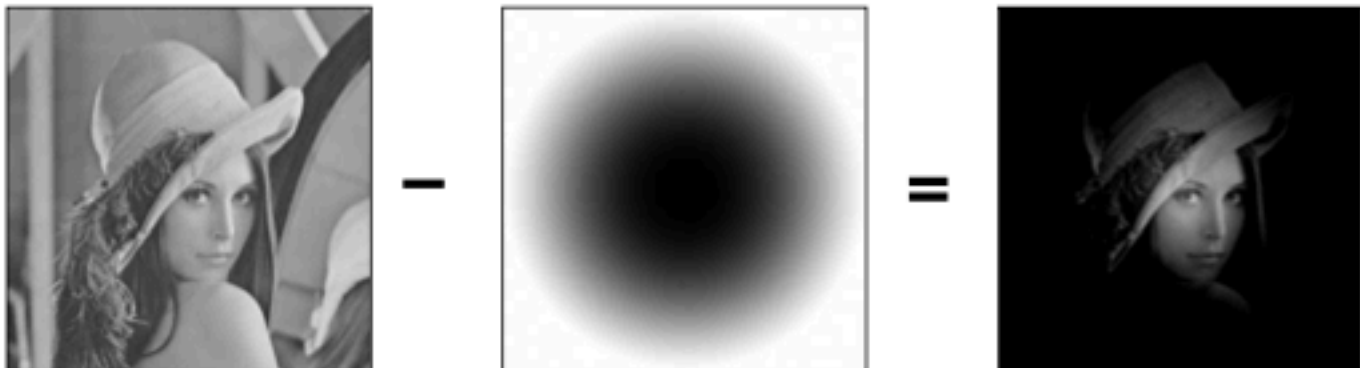
- src1: 첫 번째 영상
- alpha: 첫 번째 영상 가중치
- src2: 두 번째 영상. src1과 같은 크기 & 같은 타입
- beta: 두 번째 영상 가중치
- gamma: 결과 영상에 추가적으로 더할 값
- dst: 가중치 합 결과 영상
- dtype: 출력 영상(dst)의 타입

영상의 산술 연산

■ 뺄셈 연산

$$\text{dst}(x, y) = \text{saturate}(\text{src1}(x, y) - \text{src2}(x, y))$$

- 두 영상의 같은 위치에 존재하는 픽셀 값에 대하여 뺄셈 연산을 수행하여 결과 영상의 픽셀 값으로 설정
- 뺄셈 결과가 0보다 작으면 픽셀 값을 0으로 설정 (포화 연산)



영상의 산술 연산

■ 뺄셈 연산

```
cv2.subtract(src1, src2, dst=None, mask=None, dtype=None) -> dst
```

- src1: 첫 번째 영상 또는 스칼라
- src2: 두 번째 영상 또는 스칼라
- dst: 뺄셈 연산 결과 영상
- mask: 마스크 영상
- dtype: 출력 영상(dst)의 타입

영상의 산술 연산

■ 차이 연산

$$\text{dst}(x, y) = |\text{src1}(x, y) - \text{src2}(x, y)|$$

- 두 영상의 같은 위치에 존재하는 픽셀 값에 대하여 뺄셈 연산을 수행한 후, 그 절댓값을 결과 영상의 픽셀 값으로 설정
- 뺄셈 연산과 달리 입력 영상의 순서에 영향을 받지 않음



영상의 산술 연산

■ 차이 연산

```
cv2.absdiff(src1, src2, dst=None) -> dst
```

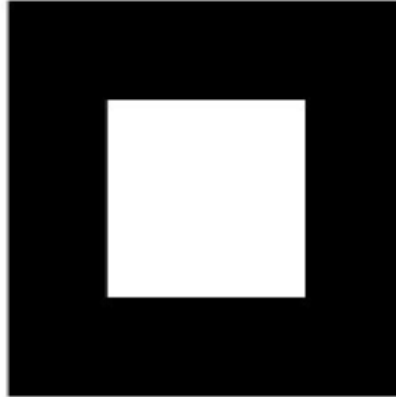
- src1: 첫 번째 영상 또는 스칼라
- src2: 두 번째 영상 또는 스칼라
- dst: 차이 연산 결과 영상(차영상)

영상의 산술 연산

src1



src2



add



addWeighted



subtract



absdiff



영상의 산술 연산

```
import sys
import numpy as np
import cv2
from matplotlib import pyplot as plt

src1 = cv2.imread('lenna256.bmp', cv2.IMREAD_GRAYSCALE)
src2 = cv2.imread('square.bmp', cv2.IMREAD_GRAYSCALE)

if src1 is None or src2 is None:
    print('Image load failed!')
    sys.exit()

dst1 = cv2.add(src1, src2, dtype=cv2.CV_8U)
dst2 = cv2.addWeighted(src1, 0.5, src2, 0.5, 0.0)
dst3 = cv2.subtract(src1, src2)
dst4 = cv2.absdiff(src1, src2)

plt.subplot(231), plt.axis('off'), plt.imshow(src1, 'gray'), plt.title('src1')
plt.subplot(232), plt.axis('off'), plt.imshow(src2, 'gray'), plt.title('src2')
plt.subplot(233), plt.axis('off'), plt.imshow(dst1, 'gray'), plt.title('add')
plt.subplot(234), plt.axis('off'), plt.imshow(dst2, 'gray'), plt.title('addWeighted')
plt.subplot(235), plt.axis('off'), plt.imshow(dst3, 'gray'), plt.title('subtract')
plt.subplot(236), plt.axis('off'), plt.imshow(dst4, 'gray'), plt.title('absdiff')
plt.show()
```

■ 비트단위 AND, OR, XOR, NOT 연산

```
cv2.bitwise_and(src1, src2, dst=None, mask=None) -> dst  
cv2.bitwise_or(src1, src2, dst=None, mask=None) -> dst  
cv2.bitwise_xor(src1, src2, dst=None, mask=None) -> dst  
cv2.bitwise_not(src1, dst=None, mask=None) -> dst
```

- src1: 첫 번째 영상 또는 스칼라
- src2: 두 번째 영상 또는 스칼라
- dst: 출력 영상
- mask: 마스크 영상
- 참고사항
 - 각각의 픽셀 값을 이진수로 표현하고, 비트(bit) 단위 논리 연산을 수행

입력 비트		논리 연산 결과			
a	b	a AND b	a OR b	a XOR b	NOT a
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

기본적인 영상 처리 기법

3) 컬러 영상과 색 공간

■ OpenCV와 컬러 영상

- 컬러 영상은 3차원 `numpy.ndarray`로 표현. `img.shape = (h, w, 3)`
- OpenCV에서는 RGB 순서가 아니라 **BGR** 순서를 기본으로 사용

■ OpenCV에서 컬러 영상 다루기

```
img1 = cv2.imread('lenna.bmp', cv2.IMREAD_COLOR)

img2 = np.zeros((480, 640, 3), np.uint8)

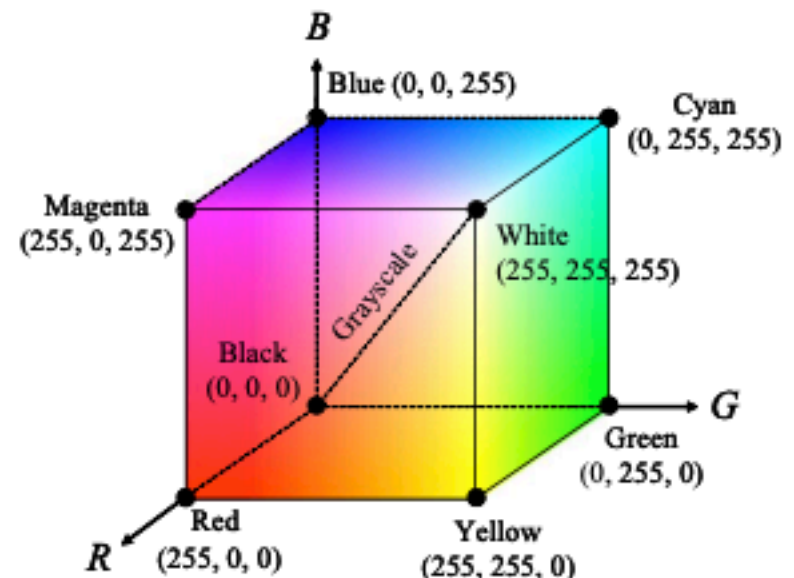
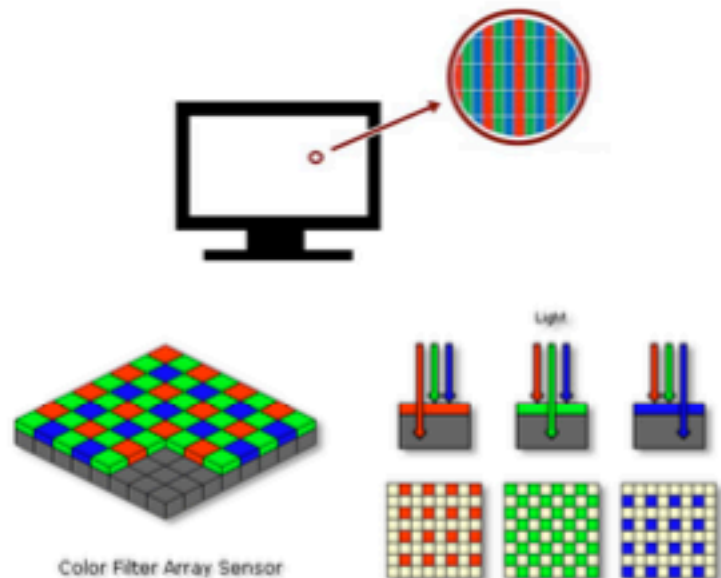
img3 = cv2.imread('lenna.bmp', cv2.IMREAD_GRAYSCALE)
img4 = cv2.cvtColor(img3, cv2.COLOR_GRAY2BGR)
```

이 경우 img4 영상의 각 픽셀은 B, G, R, 색 성분 값이 모두 같게 설정됨

컬러 영상과 색 공간

■ RGB 색 공간

- 빛의 삼원색인 빨간색(R), 녹색(G), 파란색(B)을 혼합하여 색상을 표현 (가산 혼합)
- TV & 모니터, 카메라 센서 Bayer 필터, 비트맵



컬러 영상과 색 공간

■ RGB 색상 평면 나누기

```
src = cv2.imread('candies.png', cv2.IMREAD_COLOR)

# 컬러 영상 속성 확인
print('src.shape:', src.shape) # src.shape: (480, 640, 3)
print('src.dtype:', src.dtype) # src.dtype: uint8

# RGB 색 평면 분할
b_plane, g_plane, r_plane = cv2.split(src)

cv2.imshow('src', src)
cv2.imshow('B_plane', b_plane)
cv2.imshow('G_plane', g_plane)
cv2.imshow('R_plane', r_plane)
```

→

```
b_plane = src[:, :, 0]
g_plane = src[:, :, 1]
r_plane = src[:, :, 2]
```

컬러 영상과 색 공간

```
import sys
import numpy as np
import cv2

# 컬러 영상 불러오기
src = cv2.imread('candies.png', cv2.IMREAD_COLOR)

if src is None:
    print('Image load failed!')
    sys.exit()

# 컬러 영상 속성 확인
print('src.shape:', src.shape) # src.shape: (480, 640, 3)
print('src.dtype:', src.dtype) # src.dtype: uint8

# RGB 색 평면 분할
b_plane, g_plane, r_plane = cv2.split(src)

#b_plane = src[:, :, 0]
#g_plane = src[:, :, 1]
#r_plane = src[:, :, 2]

cv2.imshow('src', src)
cv2.imshow('B_plane', b_plane)
cv2.imshow('G_plane', g_plane)
cv2.imshow('R_plane', r_plane)
cv2.waitKey()

cv2.destroyAllWindows()
```

컬러 영상과 색 공간

▪ (색상) 채널 분리

```
cv2.split(m, mv=None) -> dst
```

- m: 다채널 영상 (e.g.) (B, G, R)로 구성된 컬러 영상
- mv: 출력 영상
- dst: 출력 영상의 리스트

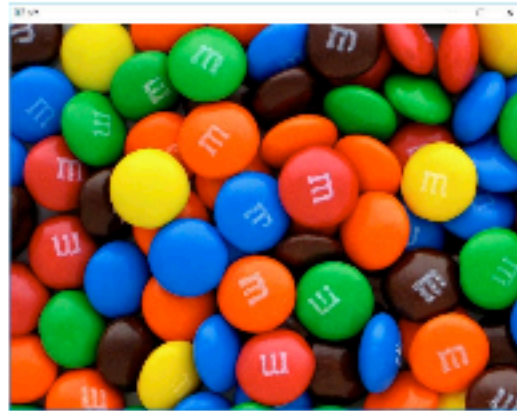
▪ (색상) 채널 결합

```
cv2.merge(mv, dst=None) -> dst
```

- mv: 입력 영상 리스트 또는 튜플
- dst: 출력 영상

컬러 영상과 색 공간

■ RGB 색상 평면



B 평면



G 평면



R 평면

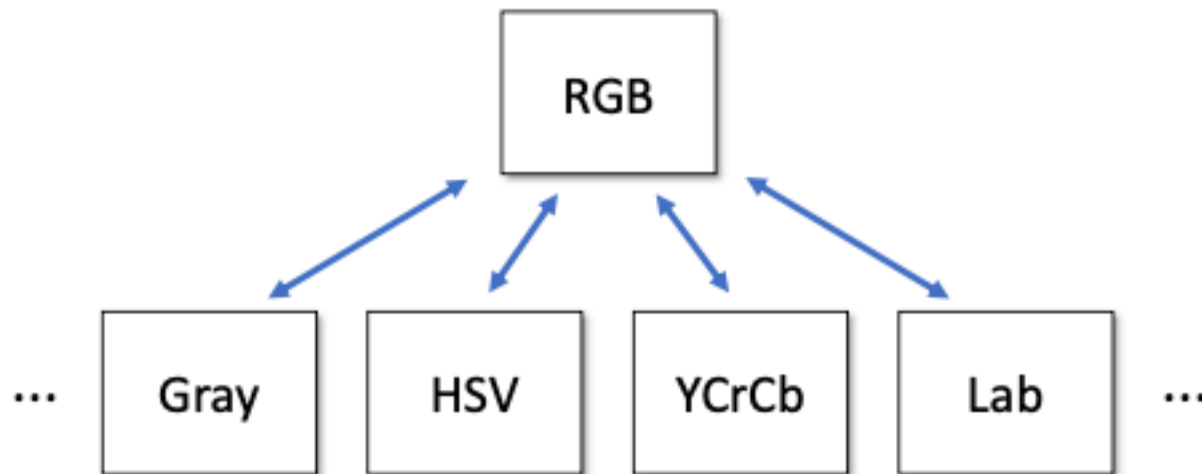
컬러 영상과 색 공간



컬러 영상과 색 공간

■ 색 공간 변환

- 영상 처리에서는 특정한 목적을 위해 RGB 색 공간을 HSV, YCrCb, Grayscale 등의 다른 색 공간으로 변환하여 처리
- OpenCV 색 공간 변환 방법
 - https://docs.opencv.org/master/de/d25/imgproc_color_conversions.html 참고



컬러 영상과 색 공간

■ 색 공간 변환 함수

```
cv2.cvtColor(src, code, dst=None, dstCn=None) -> dst
```

- src: 입력 영상
- code: 색 변환 코드 ([OpenCV 문서 페이지](#) 참고)

cv2.COLOR_BGR2GRAY / cv2.COLOR_GRAY2BGR	BGR ↔ GRAY
cv2.COLOR_BGR2RGB / cv2.COLOR_RGB2BGR	BGR ↔ RGB
cv2.COLOR_BGR2HSV / cv2.COLOR_HSV2BGR	BGR ↔ HSV
cv2.COLOR_BGR2YCrCb / cv2.COLOR_YCrCb2BGR	BGR ↔ YCrCb

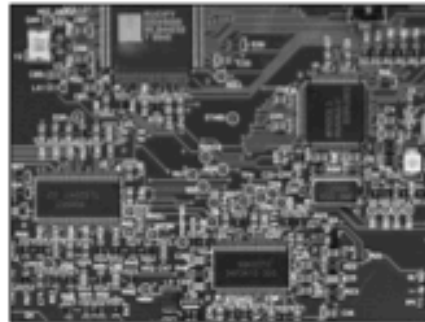
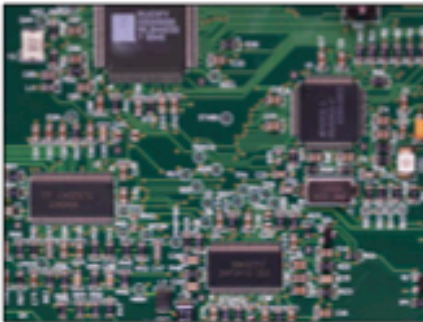
- dstCn: 결과 영상의 채널 수. 0이면 자동 결정.
- dst: 출력 영상

컬러 영상과 색 공간

- RGB 색상을 그레이스케일로 변환

$$Y = 0.299R + 0.587G + 0.114B$$

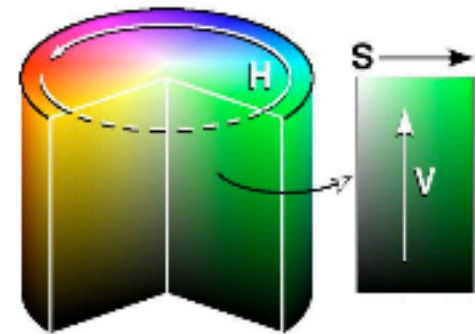
- 장점: 데이터 저장 용량 감소, 데이터 처리 속도 향상
- 단점: 색상 정보 손실



컬러 영상과 색 공간

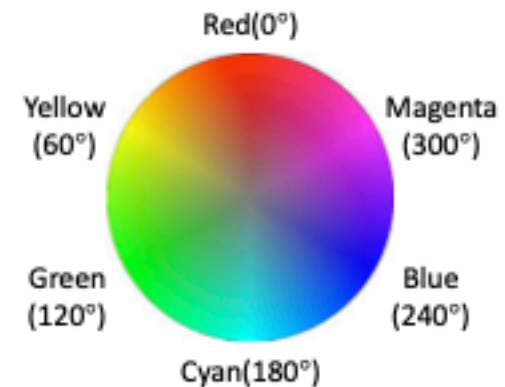
■ HSV 색 공간

- Hue: 색상, 색의 종류
- Saturation: 채도, 색의 탁하고 선명한 정도
- Value: 명도, 빛의 밝기



■ HSV 값 범위

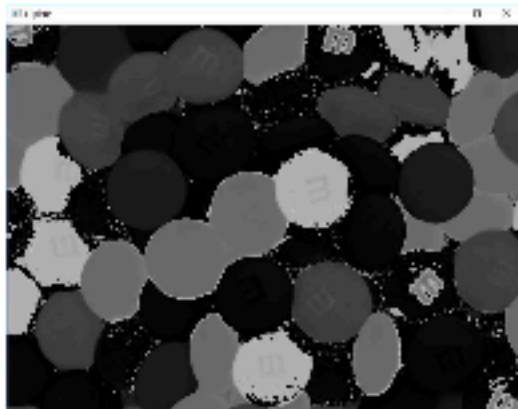
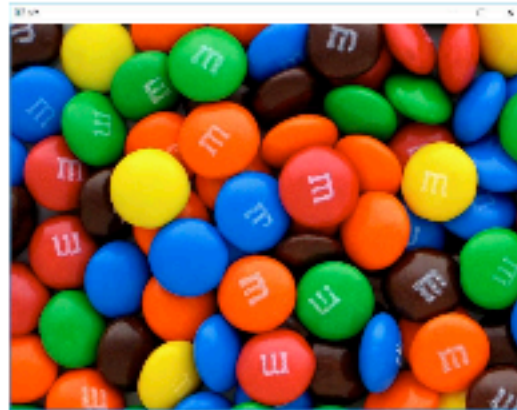
- cv2.CV_8U 영상의 경우
 - $0 \leq H \leq 179$
 - $0 \leq S \leq 255$
 - $0 \leq V \leq 255$



https://ko.wikipedia.org/wiki/HSV_%EC%83%89_%EA%B3%B5%EA%B0%84

컬러 영상과 색 공간

■ HSV 색상 평면



H 평면



S 평면



V 평면

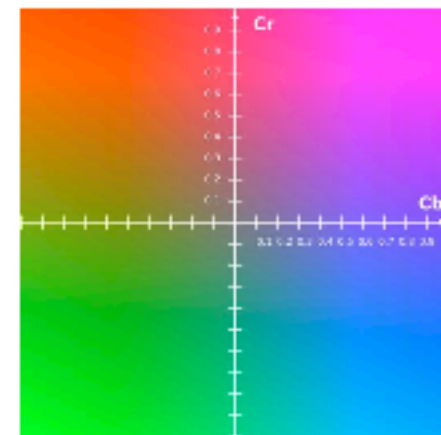
컬러 영상과 색 공간

■ YCrCb 색 공간

- PAL, NTSC, SECAM 등의 컬러 비디오 표준에 사용되는 색 공간
- 영상의 밝기 정보와 색상 정보를 따로 분리하여 부호화 (흑백 TV 호환)
- Y: 밝기 정보(luma)
- Cr, Cb: 색차(chroma)

■ YCrCb 값 범위

- cv2.CV_8U 영상의 경우
 - $0 \leq Y \leq 255$
 - $0 \leq Cr \leq 255$
 - $0 \leq Cb \leq 255$

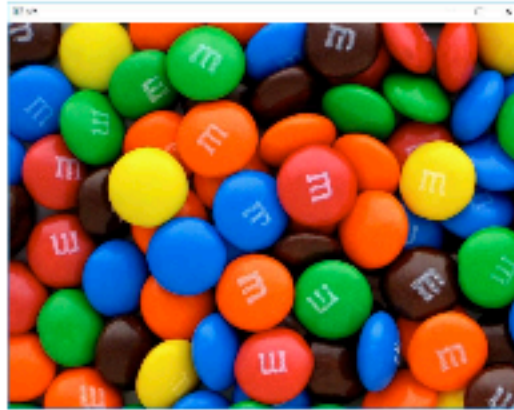


CbCr 평면 (Y=0.5 고정)

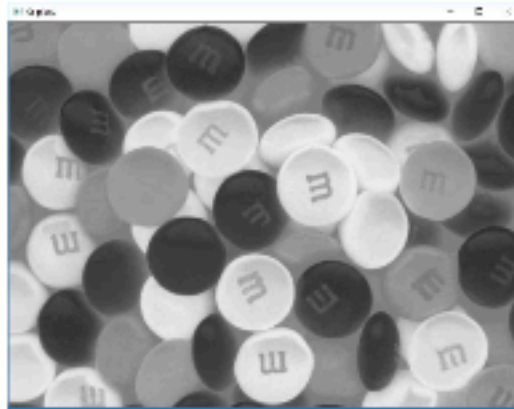
<https://ko.wikipedia.org/wiki/YUV>

컬러 영상과 색 공간

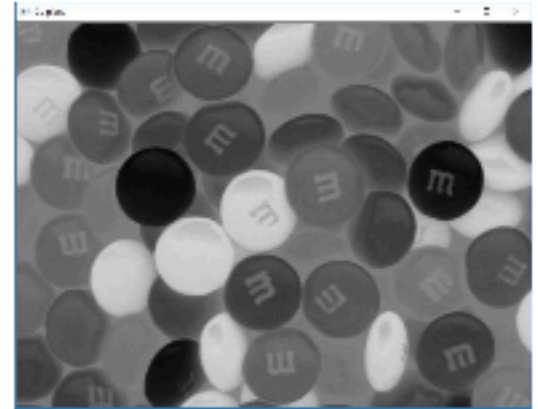
■ YCrCb 색상 평면



Y 평면



Cr 평면



Cb 평면

기본적인 영상 처리 기법

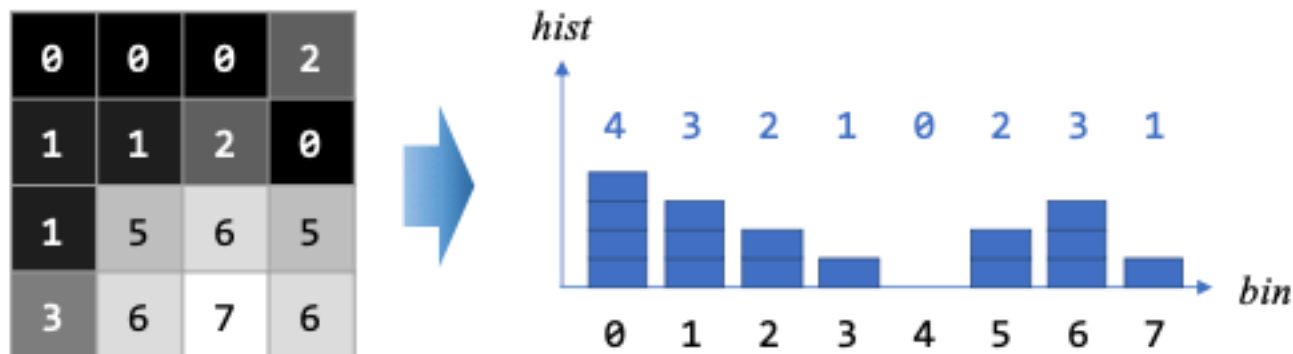
4) 히스토그램 분석

히스토그램 분석

■ 히스토그램(Histogram)

- 영상의 픽셀 값 분포를 그래프의 형태로 표현한 것
- 예를 들어 그레이스케일 영상에서 각 그레이스케일 값에 해당하는 픽셀의 개수를 구하고, 이를 막대 그래프의 형태로 표현

$$h(g) = N_g$$



히스토그램 분석

- 정규화된 히스토그램(Normalized histogram)
 - 각 픽셀의 개수를 영상 전체 픽셀 개수로 나누어준 것
 - 해당 그레이스케일 값을 갖는 픽셀이 나타날 **확률**

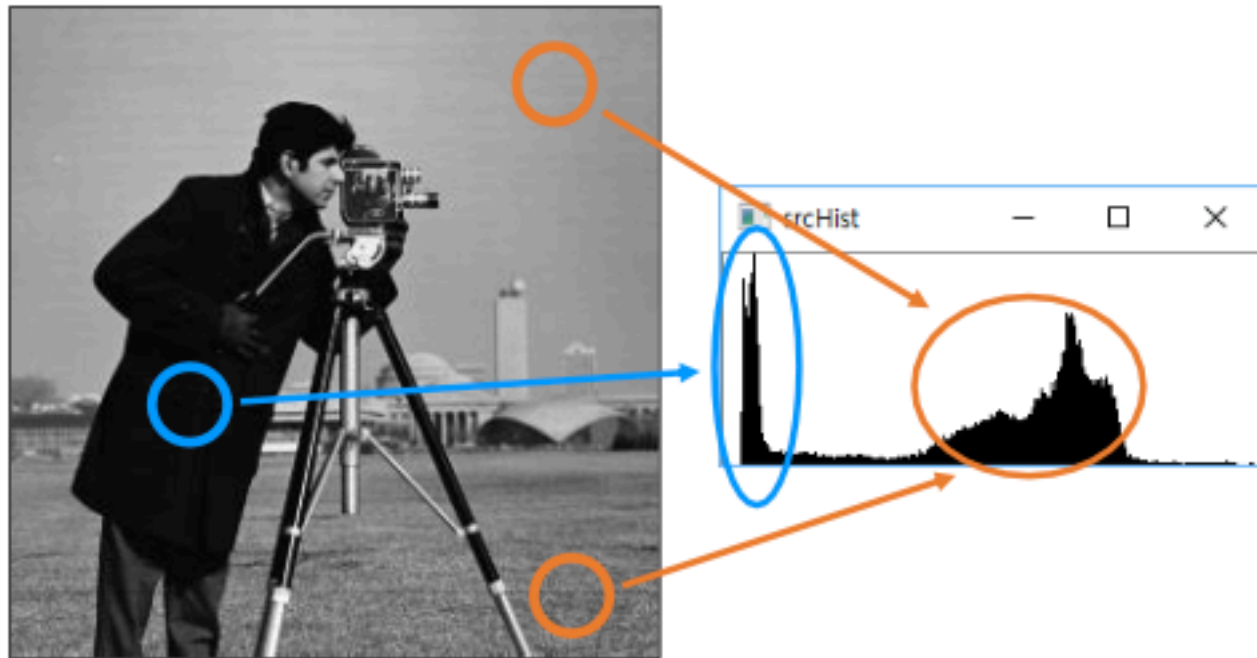
$$p(g) = \frac{N_g}{w \times h} \quad \Rightarrow \quad \sum_{g=0}^{L-1} p(g) = 1$$

0	0	0	2
1	1	2	0
1	5	6	5
3	6	7	6

g	0	1	2	3	4	5	6	7
$h(g)$	4	3	2	1	0	2	3	1
$p(g)$	$\frac{4}{16}$	$\frac{3}{16}$	$\frac{2}{16}$	$\frac{1}{16}$	0	$\frac{2}{16}$	$\frac{3}{16}$	$\frac{1}{16}$

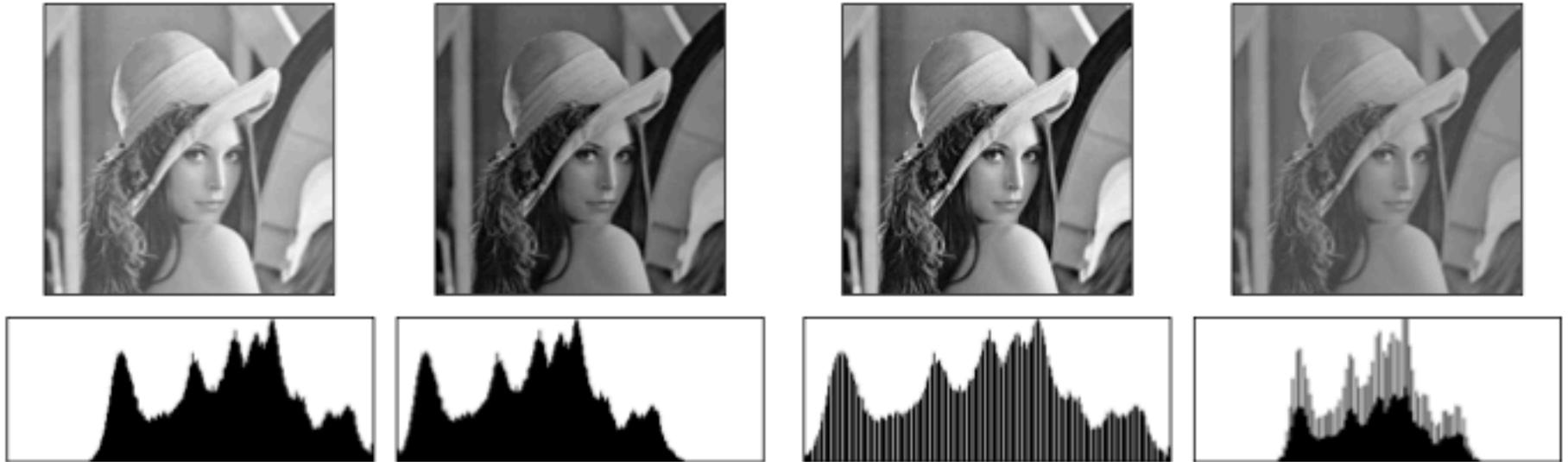
히스토그램 분석

- 영상과 히스토그램의 관계



히스토그램 분석

■ 영상과 히스토그램의 관계



히스토그램 분석

■ 히스토그램 구하기

```
cv2.calcHist(images, channels, mask, histSize, ranges, hist=None,  
             accumulate=None) -> hist
```

- images: 입력 영상 리스트
- channels: 히스토그램을 구할 채널을 나타내는 리스트
- mask: 마스크 영상. 입력 영상 전체에서 히스토그램을 구하려면 **None** 지정.
- histSize: 히스토그램 각 차원의 크기(빈(bin)의 개수)를 나타내는 리스트
- ranges: 히스토그램 각 차원의 최솟값과 최댓값으로 구성된 리스트
- hist: 계산된 히스토그램 (**numpy.ndarray**)
- accumulate: 기존의 hist 히스토그램에 누적하려면 True, 새로 만들려면 False.

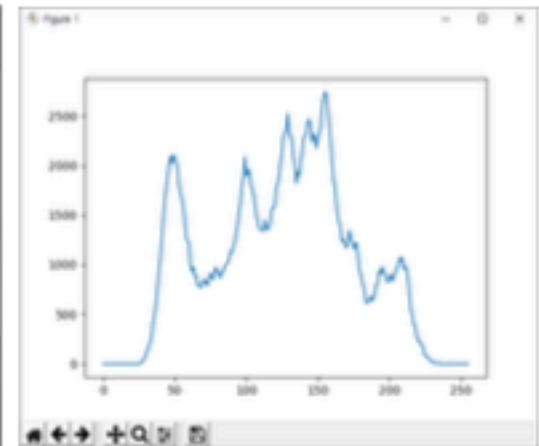
히스토그램 분석

- 그레이스케일 영상의 히스토그램 구하기

```
src = cv2.imread('lenna.bmp', cv2.IMREAD_GRAYSCALE)

hist = cv2.calcHist([src], [0], None, [256], [0, 256])

plt.plot(hist)
plt.show()
```



히스토그램 분석

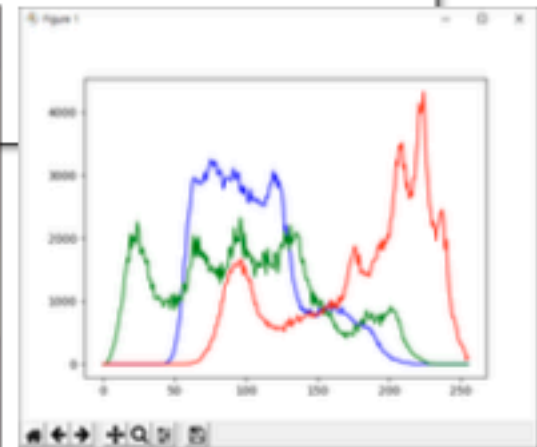
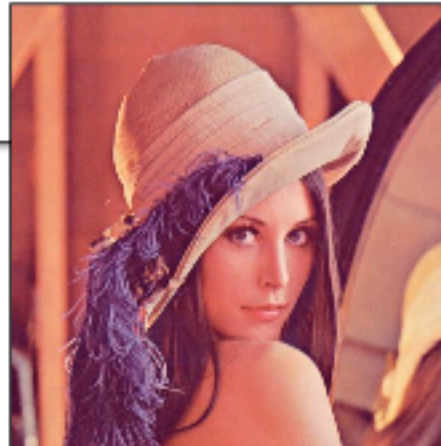
■ 컬러 영상의 히스토그램 구하기

```
src = cv2.imread('lenna.bmp')

colors = ['b', 'g', 'r']
bgr_planes = cv2.split(src)

for (p, c) in zip(bgr_planes, colors):
    hist = cv2.calcHist([p], [0], None, [256], [0, 256])
    plt.plot(hist, color=c)

plt.show()
```



히스토그램 분석

- OpenCV 그리기 함수로 그레이스케일 영상의 히스토그램 나타내기

```
def getGrayHistImage(hist):  
    imgHist = np.full((100, 256), 255, dtype=np.uint8)  
  
    histMax = np.max(hist)  
    for x in range(256):  
        pt1 = (x, 100)  
        pt2 = (x, 100 - int(hist[x, 0] * 100 / histMax))  
        cv2.line(imgHist, pt1, pt2, 0)  
  
    return imgHist  
  
src = cv2.imread('lenna.bmp', cv2.IMREAD_GRAYSCALE)  
  
hist = cv2.calcHist([src], [0], None, [256], [0, 256])  
  
histImg = getGrayHistImage(hist)
```



히스토그램 분석

```
import sys
import numpy as np
import matplotlib.pyplot as plt
import cv2

def getGrayHistImage(hist):
    imgHist = np.full((100, 256), 255, dtype=np.uint8)

    histMax = np.max(hist)
    for x in range(256):
        pt1 = (x, 100)
        pt2 = (x, 100 - int(hist[x, 0] * 100 / histMax))
        cv2.line(imgHist, pt1, pt2, 0)

    return imgHist

src = cv2.imread('lenna.bmp', cv2.IMREAD_GRAYSCALE)

if src is None:
    print('Image load failed!')
    sys.exit()

hist = cv2.calcHist([src], [0], None, [256], [0, 256])
histImg = getGrayHistImage(hist)

cv2.imshow('src', src)
cv2.imshow('histImg', histImg)
cv2.waitKey()

cv2.destroyAllWindows()
```

기본적인 영상 처리 기법

5) 영상의 명암비 조절

영상의 명암비 조절

■ 명암비(Contrast)란?

- 밝은 곳과 어두운 곳 사이에 드러나는 밝기 정도의 차이
- 컨트라스트, 대비



명암비가 낮은 영상



명암비가 높은 영상

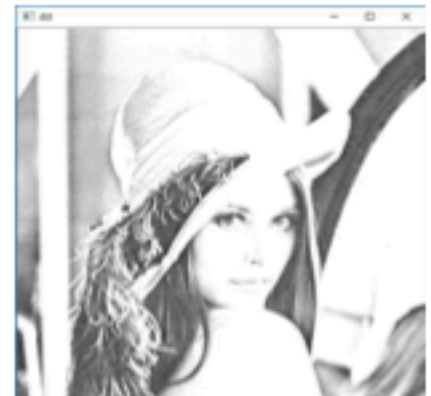
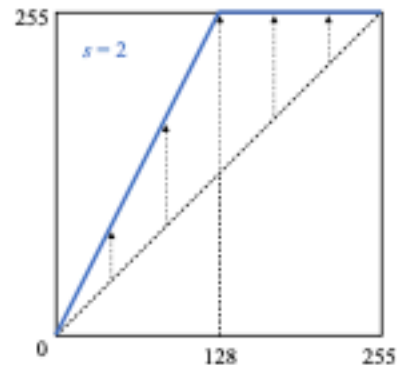
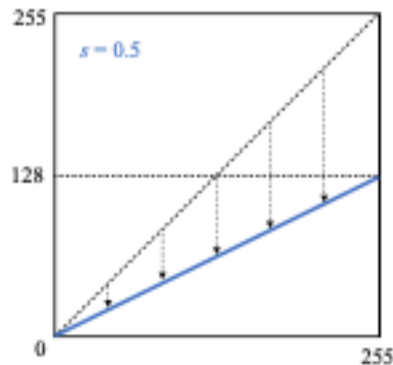
영상의 명암비 조절

■ 기본적인 명암비 조절 함수

$$\text{dst}(x, y) = \text{saturate}(s \cdot \text{src}(x, y))$$



$s = 0.5$

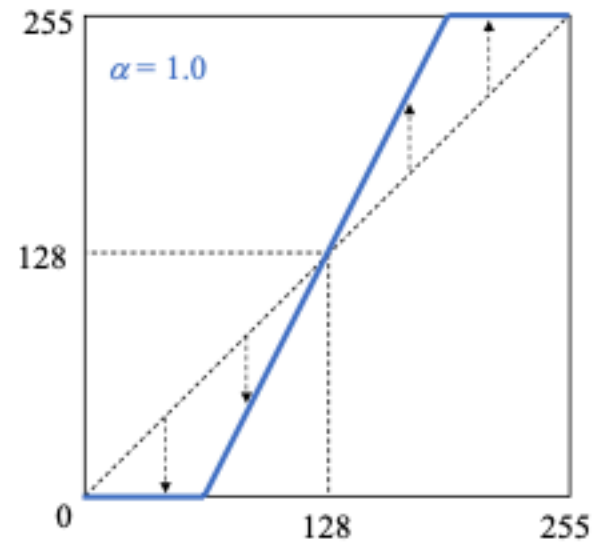
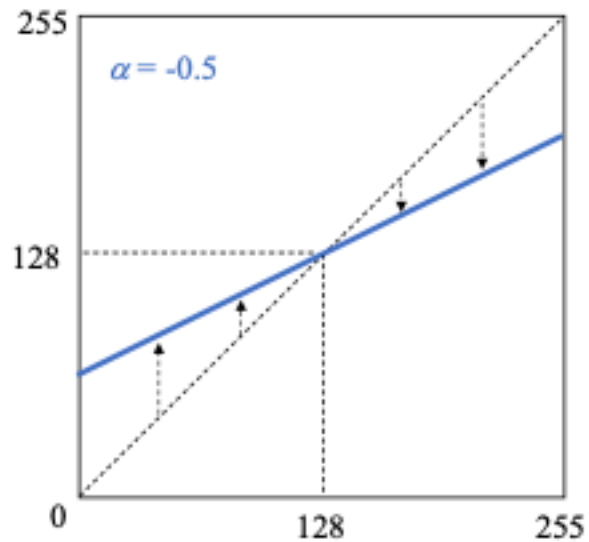


$s = 2.0$

영상의 명암비 조절

- 효과적인 명암비 조절 함수

$$\text{dst}(x, y) = \text{saturate}(\text{src}(x, y) + (\text{src}(x, y) - 128) \cdot \alpha)$$

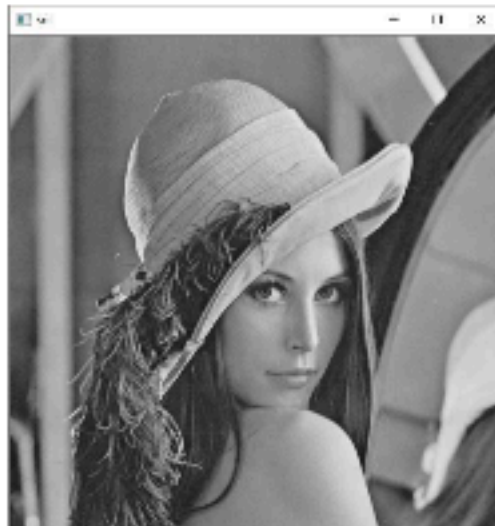


영상의 명암비 조절

■ 기본적인 명암비 조절 예제

```
src = cv2.imread('lenna.bmp', cv2.IMREAD_GRAYSCALE)

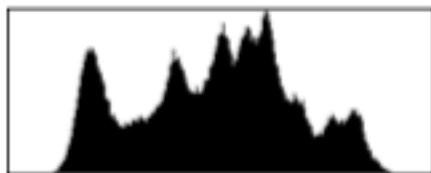
alpha = 1.0
dst = np.clip((1+alpha)*src - 128*alpha, 0, 255).astype(np.uint8)
```



영상의 명암비 조절

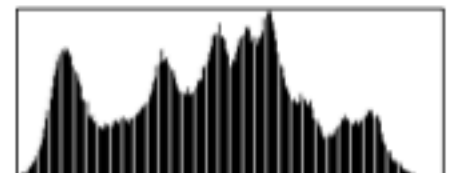
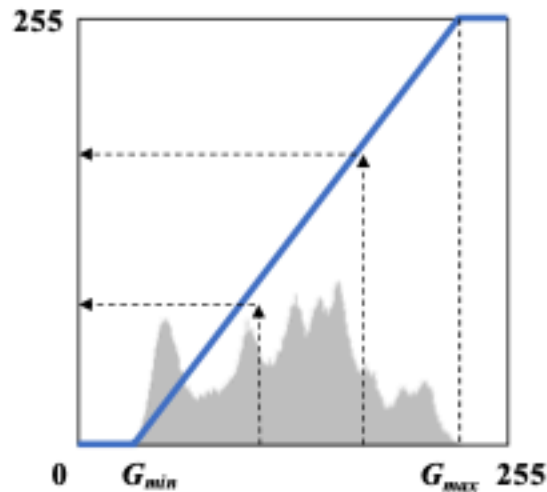
■ 히스토그램 스트레칭(Histogram stretching)

- 영상의 히스토그램이 그레이스케일 전 구간에서 걸쳐 나타나도록 변경하는 선형 변환 기법



G_{min}

G_{max}



$G_{min} = 0$

$G_{max} = 255$

영상의 자동 명암비 조절

■ 정규화 함수

```
cv2.normalize(src, dst, alpha=None, beta=None, norm_type=None, dtype=None,  
             mask=None) -> dst
```

- src: 입력 영상
- dst: 결과 영상
- alpha: (노름 정규화인 경우) 목표 노름 값,
(원소 값 범위 정규화인 경우) 최솟값
- beta: (원소 값 범위 정규화인 경우) 최댓값
- norm_type: 정규화 타입. NORM_INF, NORM_L1, NORM_L2, NORM_MINMAX.
- dtype: 결과 영상의 타입
- mask: 마스크 영상

영상의 자동 명암비 조절

- 히스토그램 스트레칭을 이용한 명암비 자동 조절

```
src = cv2.imread('Hawkes.jpg', cv2.IMREAD_GRAYSCALE)  
dst = cv2.normalize(src, None, 0, 255, cv2.NORM_MINMAX)
```



기본적인 영상 처리 기법

6) 히스토그램 평활화

히스토그램 평활화

- 히스토그램 평활화(Histogram equalization)

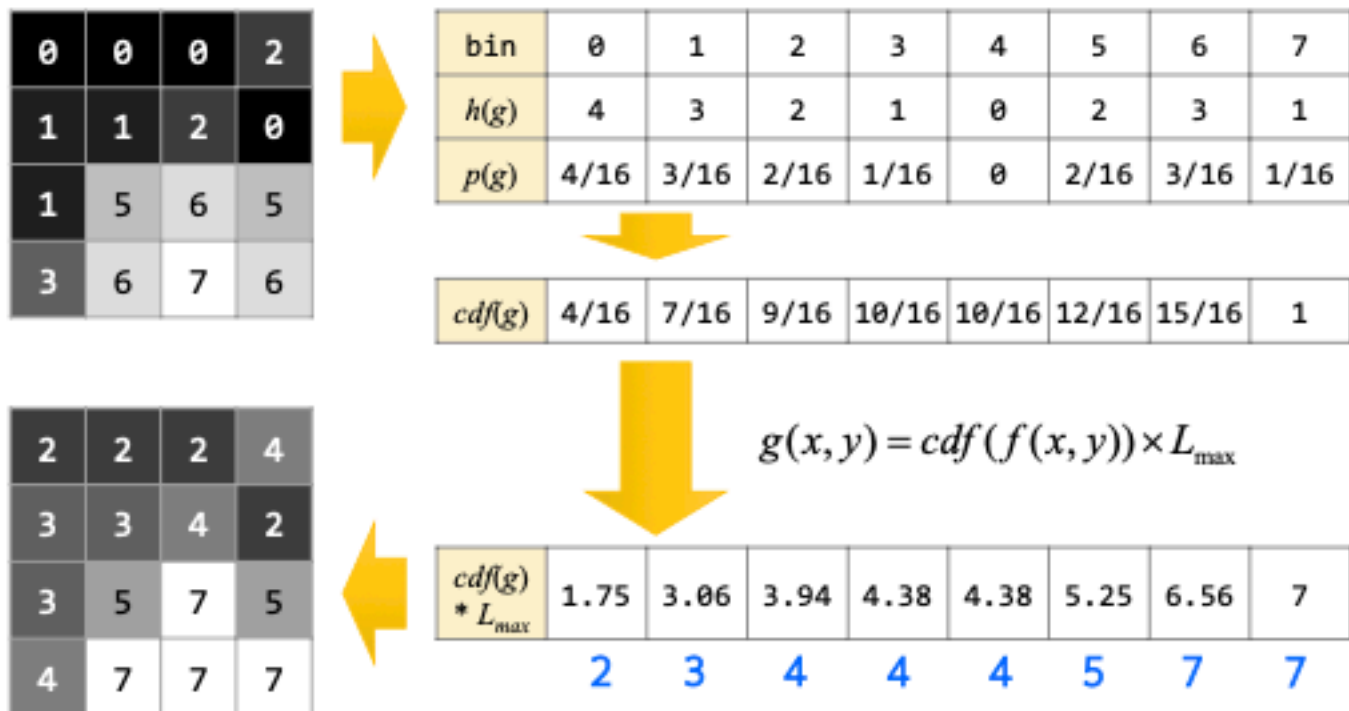
- 히스토그램이 그레이스케일 전체 구간에서 균일한 분포로 나타나도록 변경하는 명암비 향상 기법
- 히스토그램 균등화, 균일화, 평탄화

- 히스토그램 평활화를 위한 변환 함수 구하기

- 히스토그램 함수 구하기: $h(g) = N_g$
- 정규화된 히스토그램 함수 구하기: $p(g) = \frac{h(g)}{w \times h}$
- 누적 분포 함수(cdf) 구하기: $cdf(g) = \sum_{0 \leq i < g} p(i)$
- 변환 함수: $dst(x, y) = round(cdf(src(x, y)) \times L_{max})$

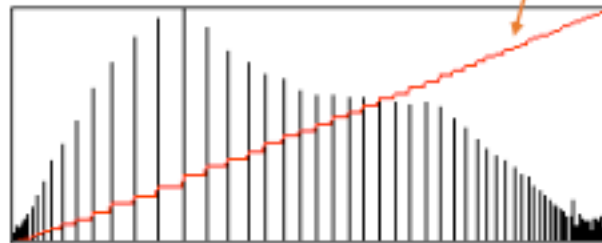
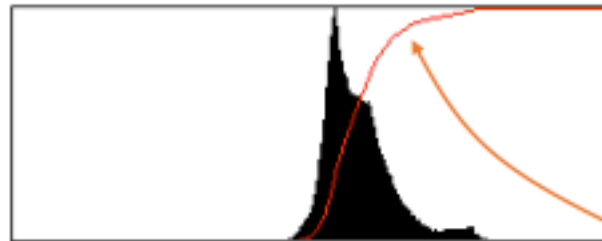
히스토그램 평활화

■ 히스토그램 평활화 계산 방법



히스토그램 평활화

- 히스토그램 평활화와 히스토그램 누적 분포 함수와의 관계



히스토그램
누적 분포 함수

https://en.wikipedia.org/wiki/Histogram_equalization

히스토그램 평활화

■ 히스토그램 평활화

```
cv2.equalizeHist(src, dst=None) -> dst
```

- src: 입력 영상. 그레이스케일 영상.
- dst: 결과 영상.

히스토그램 평활화

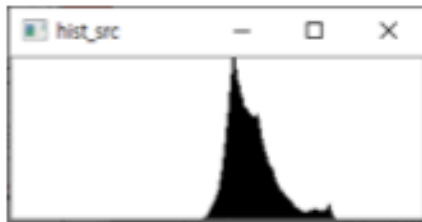
■ 히스토그램 평활화 예제

```
src = cv2.imread('Hawkes.jpg', cv2.IMREAD_GRAYSCALE)  
dst = cv2.equalizeHist(src)
```

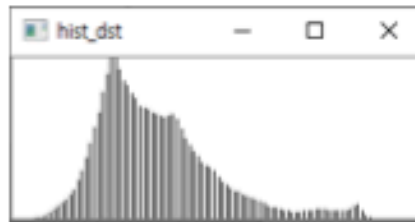


히스토그램 평활화

■ 히스토그램 스트레칭과 평활화 비교



입력 영상



히스토그램 스트레칭

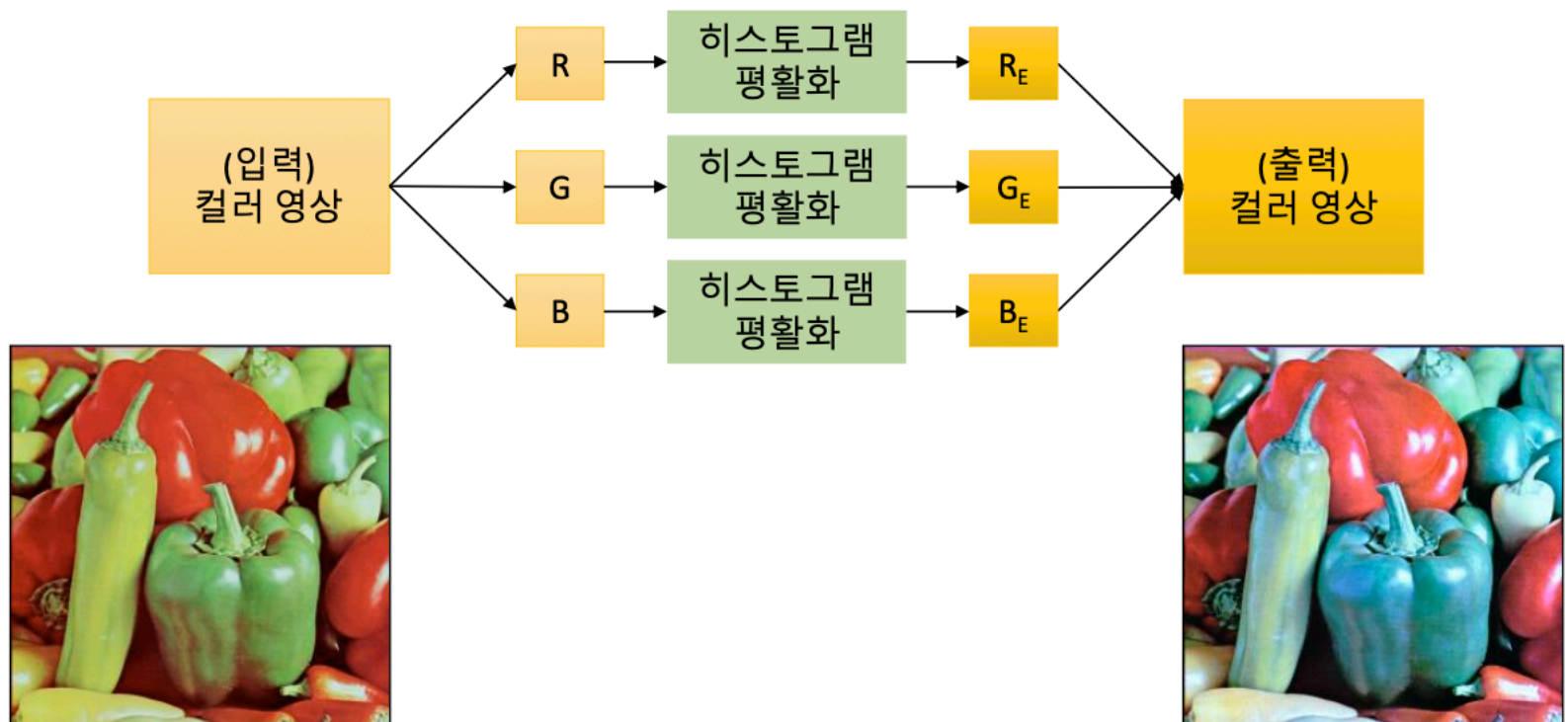


히스토그램 평활화

컬러 영상의 히스토그램 평활화

■ 컬러 히스토그램 평활화

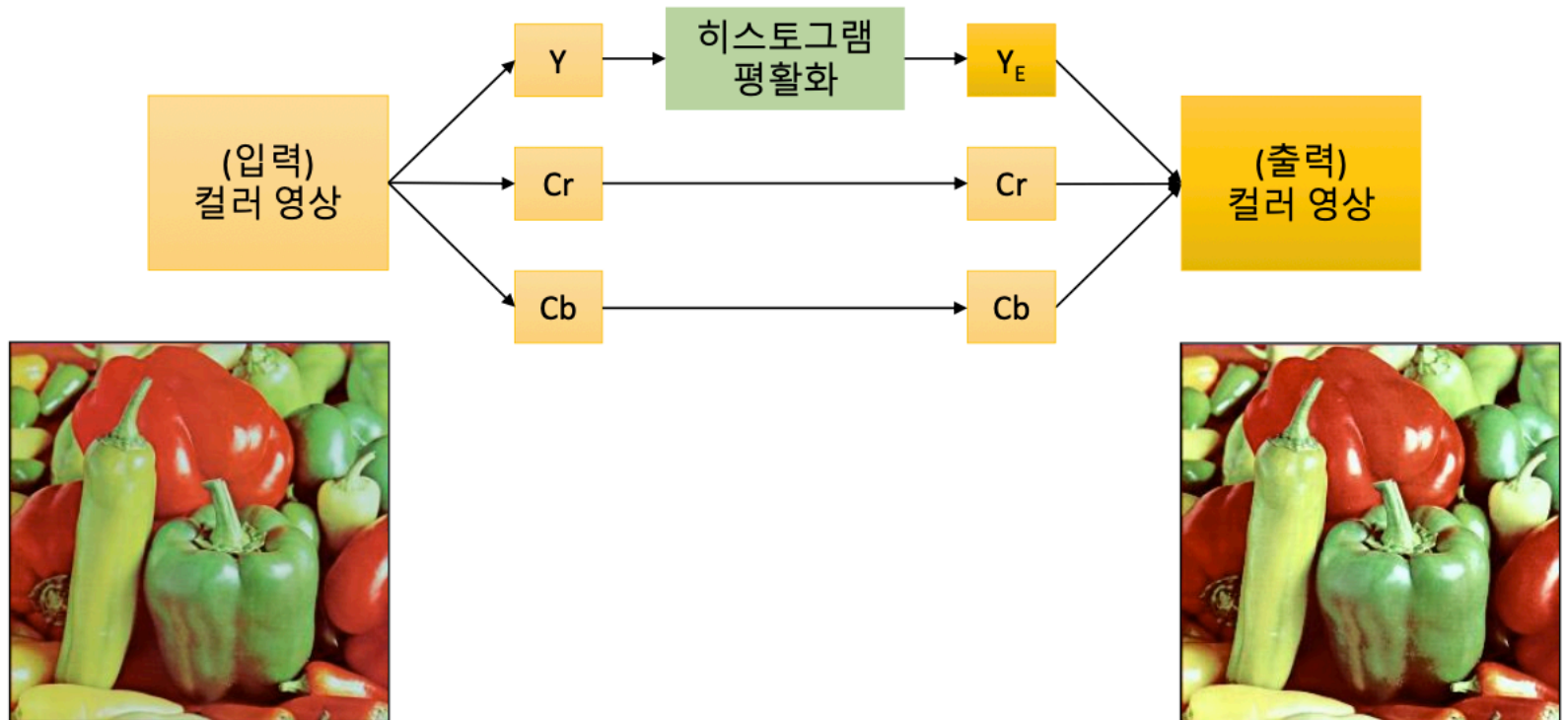
- 직관적 방법: R, G, B 각 색 평면에 대해 히스토그램 평활화



컬러 영상의 히스토그램 평활화

■ 컬러 히스토그램 평활화

- 밝기 성분에 대해서만 히스토그램 평활화 수행 (색상 성분은 불변)



컬러 영상의 히스토그램 평활화

■ 컬러 영상의 히스토그램 평활화

```
src = cv2.imread('field.bmp')

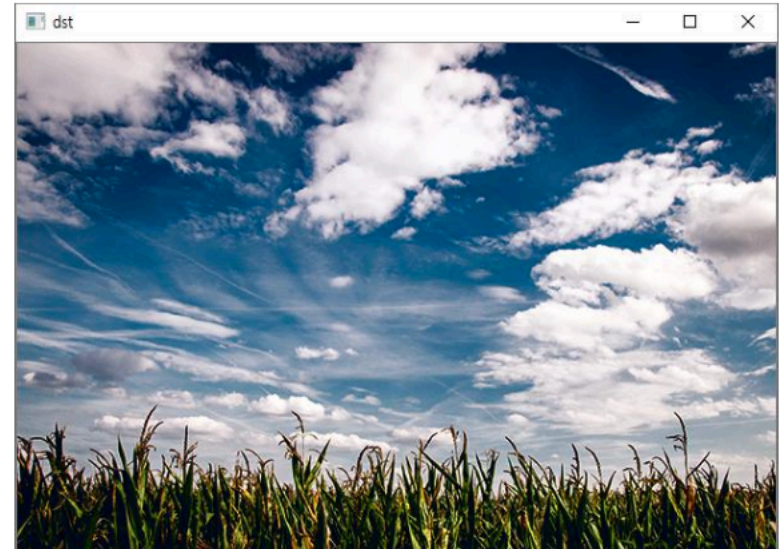
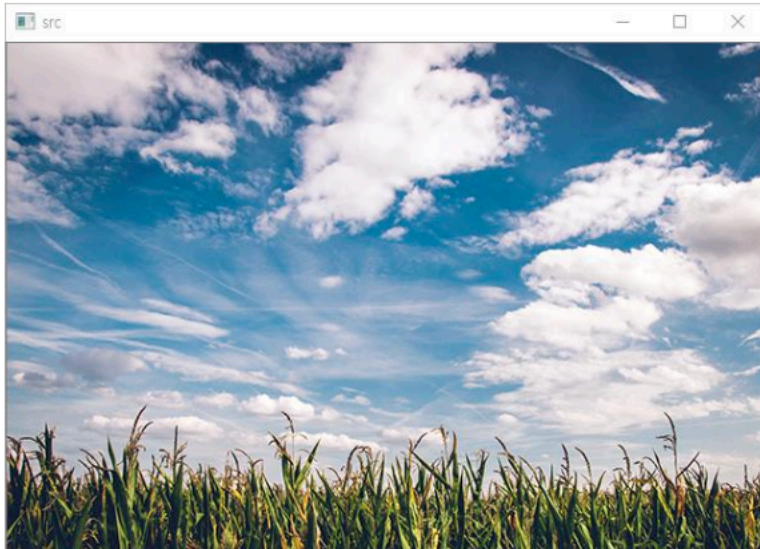
src_ycrb = cv2.cvtColor(src, cv2.COLOR_BGR2YCrCb)
ycrcb_planes = cv2.split(src_ycrb)

# 밝기 성분에 대해서만 히스토그램 평활화 수행
ycrcb_planes[0] = cv2.equalizeHist(ycrcb_planes[0])

dst_ycrb = cv2.merge(ycrcb_planes)
dst = cv2.cvtColor(dst_ycrb, cv2.COLOR_YCrCb2BGR)
```

컬러 영상의 히스토그램 평활화

■ 컬러 영상의 히스토그램 평활화 결과



기본적인 영상 처리 기법

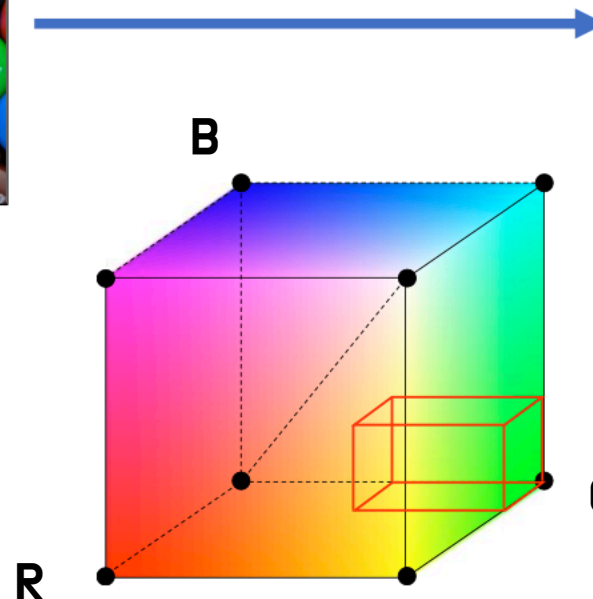
7) 특정 색상 영역 추출

특정 색상 영역 추출

■ RGB 색 공간에서 녹색 영역 추출하기



$$\begin{aligned} 0 &\leq R \leq 100 \\ 128 &\leq G \leq 255 \\ 0 &\leq B \leq 100 \end{aligned}$$



Mask 영상



흰색은 녹색
검은색은 else

특정 색상 영역 추출

- HSV 색 공간에서 녹색 영역 추출하기

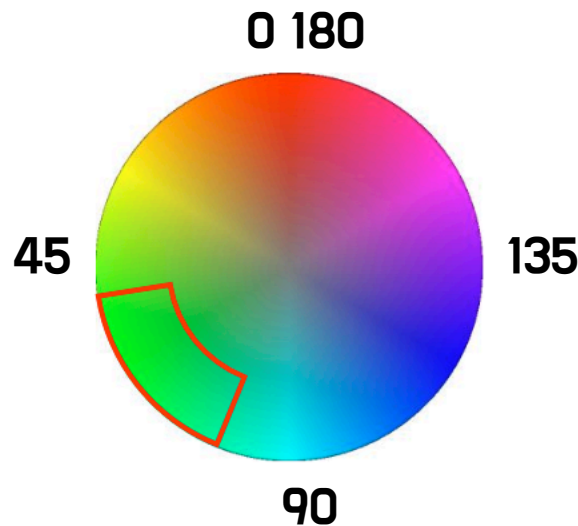


$$\begin{aligned} 50 &\leq H \leq 80 \\ 150 &\leq S \leq 255 \\ 0 &\leq V \leq 255 \end{aligned}$$

Mask 영상



H : 색의 종류
S : 색상의 선명도
V : 밝기



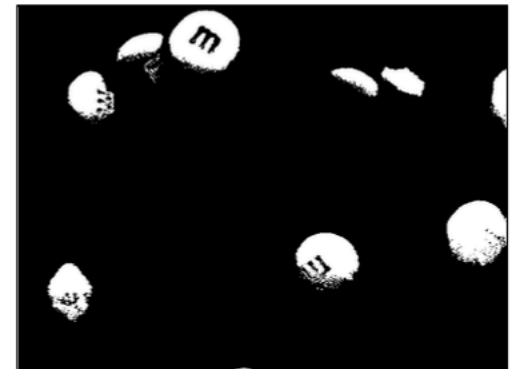
흰색은 녹색
검은색은 else

특정 색상 영역 추출

- RGB 색 공간에서 녹색 영역 추출하기



$$\begin{aligned}0 &\leq R \leq 100 \\128 &\leq G \leq 255 \\0 &\leq B \leq 100\end{aligned}$$



특정 색상 영역 추출

■ HSV 색 공간에서 녹색 영역 추출하기



$$\begin{aligned} 50 &\leq H \leq 80 \\ 150 &\leq S \leq 255 \\ 0 &\leq V \leq 255 \end{aligned}$$



특정 색상 영역 추출

■ 특정 범위 안에 있는 행렬 원소 검출

```
cv2.inRange(src, lowerb, upperb, dst=None) -> dst
```

- src: 입력 행렬
- lowerb: 하한 값 행렬 또는 스칼라
- upperb: 상한 값 행렬 또는 스칼라
- dst: 입력 영상과 같은 크기의 마스크 영상. (numpy.uint8)
범위 안에 들어가는 픽셀은 255, 나머지는 0으로 설정.

- 단일 채널: $\text{dst}(I) = \text{lowerb}(I)_0 \leq \text{src}(I)_0 \leq \text{upperb}(I)_0$
- 다중 채널: $\text{dst}(I) = \text{lowerb}(I)_0 \leq \text{src}(I)_0 \leq \text{upperb}(I)_0 \wedge$
 $\text{lowerb}(I)_1 \leq \text{src}(I)_1 \leq \text{upperb}(I)_1 \wedge$
...

특정 색상 영역 추출

```
import sys
import numpy as np
import cv2
```

```
src = cv2.imread('candies.png')
#src = cv2.imread('candies2.png')
```

```
if src is None:
    print('Image load failed!')
    sys.exit()
```

```
src_hsv = cv2.cvtColor(src, cv2.COLOR_BGR2HSV)
```

$0 \leq R \leq 100$

$128 \leq G \leq 255$

$0 \leq B \leq 100$

```
dst1 = cv2.inRange(src, (0, 128, 0), (100, 255, 100))
dst2 = cv2.inRange(src_hsv, (50, 150, 0), (80, 255, 255))
```

```
cv2.imshow('src', src)
cv2.imshow('dst1', dst1)
cv2.imshow('dst2', dst2)
cv2.waitKey()
```

$50 \leq H \leq 80$
 $150 \leq S \leq 255$
 $0 \leq V \leq 255$

```
cv2.destroyAllWindows()
```

특정 색상 영역 추출

■ 트랙바를 이용한 특정 색상 영역 추출

```
src = cv2.imread('candies.png')
src_hsv = cv2.cvtColor(src, cv2.COLOR_BGR2HSV)

def on_trackbar(pos):
    hmin = cv2.getTrackbarPos('H_min', 'dst')
    hmax = cv2.getTrackbarPos('H_max', 'dst')

    dst = cv2.inRange(src_hsv, (hmin, 150, 0), (hmax, 255, 255))
    cv2.imshow('dst', dst)

cv2.imshow('src', src)
cv2.namedWindow('dst')

cv2.createTrackbar('H_min', 'dst', 50, 179, on_trackbar)
cv2.createTrackbar('H_max', 'dst', 80, 179, on_trackbar)
on_trackbar(0)
cv2.waitKey()
```

특정 색상 영역 추출

```
import sys
import numpy as np
import cv2

src = cv2.imread('candies.png')

if src is None:
    print('Image load failed!')
    sys.exit()

src_hsv = cv2.cvtColor(src, cv2.COLOR_BGR2HSV)

def on_trackbar(pos):
    hmin = cv2.getTrackbarPos('H_min', 'dst')
    hmax = cv2.getTrackbarPos('H_max', 'dst')

    dst = cv2.inRange(src_hsv, (hmin, 150, 0), (hmax, 255, 255))
    cv2.imshow('dst', dst)

cv2.imshow('src', src)
cv2.namedWindow('dst')

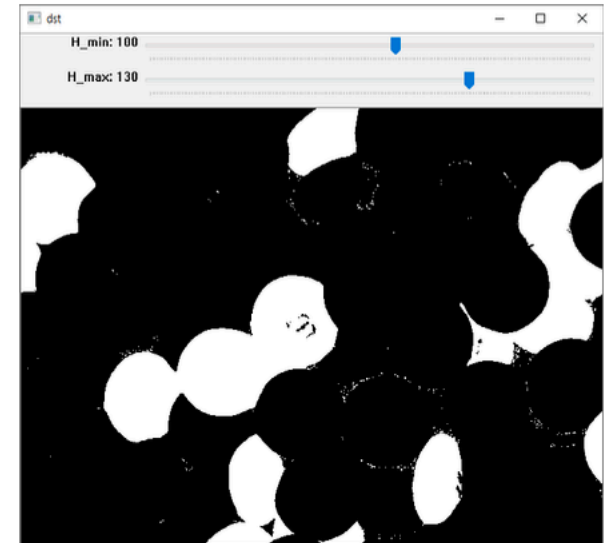
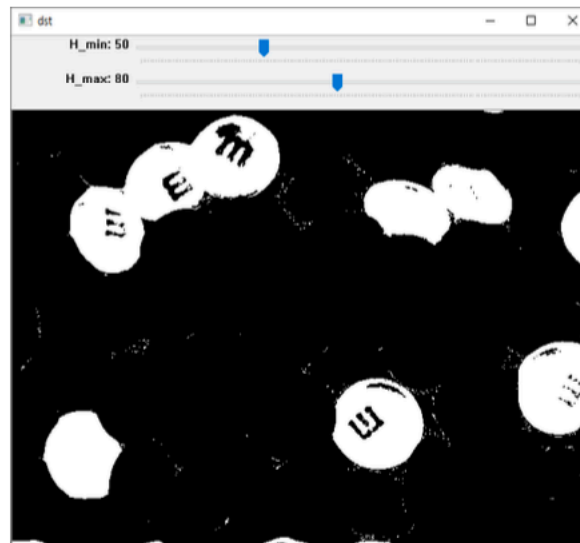
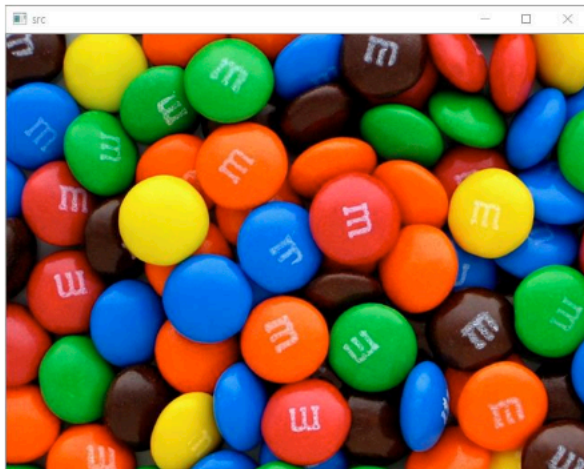
cv2.createTrackbar('H_min', 'dst', 50, 179, on_trackbar)
cv2.createTrackbar('H_max', 'dst', 80, 179, on_trackbar)
on_trackbar(0)

cv2.waitKey()

cv2.destroyAllWindows()
```

특정 색상 영역 추출

■ 트랙바를 이용한 특정 색상 영역 추출 실행 결과



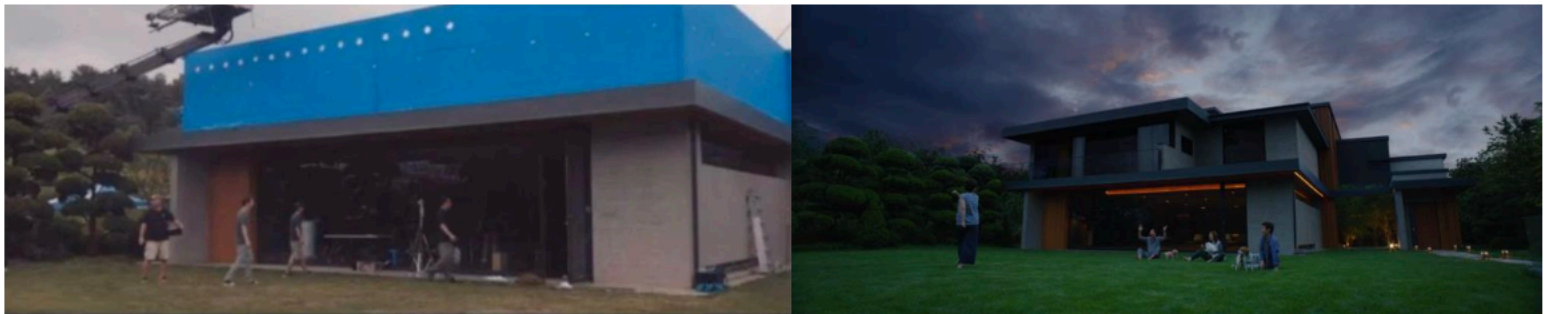
기본적인 영상 처리 기법

8) 실전 코딩 : 크로마 키 합성

[실전 코딩] 크로마 키 합성

■ 크로마 키(Chroma key) 합성이란?

- 녹색 또는 파란색 배경에서 촬영한 영상에 다른 배경 영상을 합성하는 기술



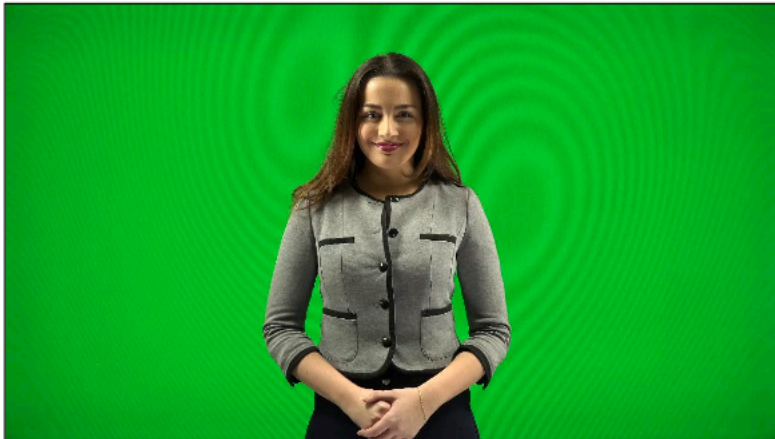
■ 구현 할 기능

- 녹색 스크린 영역 추출하기
- 녹색 영역에 다른 배경 영상을 합성하여 저장하기
- 스페이스바를 이용하여 크로마 키 합성 동작 제어하기

[실전 코딩] 크로마 키 합성

■ 녹색 스크린 영역 추출하기

- 크로마 키 영상을 HSV 색 공간으로 변환
- `cv2.inRange()` 함수를 사용하여 $50 \leq H \leq 80$, $150 \leq S \leq 255$, $0 \leq V \leq 255$ 범위의 영역을 검출



[실전 코딩] 크로마 키 합성

- 녹색 영역에 다른 배경 영상을 합성하기
 - 마스크 연산을 지원하는 `cv2.copyTo()` 함수 사용



[실전 코딩] 크로마 키 합성

```
import sys
import numpy as np
import cv2

# 녹색 배경 동영상
cap1 = cv2.VideoCapture('woman.mp4')

if not cap1.isOpened():
    print('video open failed!')
    sys.exit()

# 비오는 배경 동영상
cap2 = cv2.VideoCapture('raining.mp4')

if not cap2.isOpened():
    print('video open failed!')
    sys.exit()

# 두 동영상의 크기, FPS는 같다고 가정
frame_cnt1 = round(cap1.get(cv2.CAP_PROP_FRAME_COUNT))
frame_cnt2 = round(cap2.get(cv2.CAP_PROP_FRAME_COUNT))
print('frame_cnt1:', frame_cnt1)
print('frame_cnt2:', frame_cnt2)

fps = cap1.get(cv2.CAP_PROP_FPS)
delay = int(1000 / fps)

# 합성 여부 플래그
do_composit = False
```

[실전 코딩] 크로마 키 합성

```
# 전체 동영상 재생
while True:
    ret1, frame1 = cap1.read()

    if not ret1:
        break

    # do_composit 플래그가 True일 때에만 합성
    if do_composit:
        ret2, frame2 = cap2.read()

        if not ret2:
            break

        # HSV 색 공간에서 녹색 영역을 검출하여 합성
        hsv = cv2.cvtColor(frame1, cv2.COLOR_BGR2HSV)
        mask = cv2.inRange(hsv, (50, 150, 0), (70, 255, 255))
        cv2.copyTo(frame2, mask, frame1)

    cv2.imshow('frame', frame1)
    key = cv2.waitKey(delay)

    # 스페이스바를 누르면 do_composit 플래그를 변경
    if key == ord(' '):
        do_composit = not do_composit
    elif key == 27:
        break

cap1.release()
cap2.release()
cv2.destroyAllWindows()
```

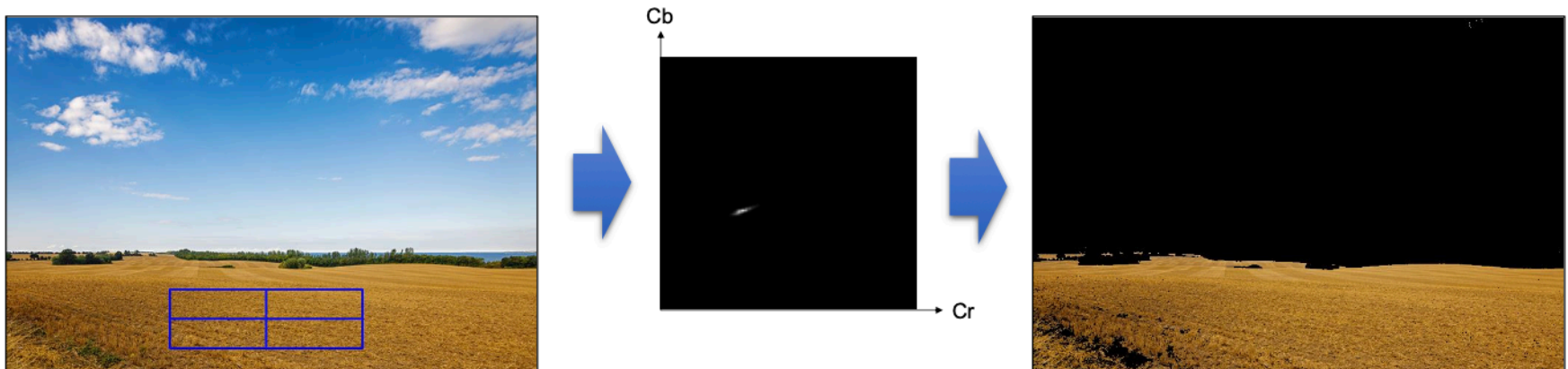
기본적인 영상 처리 기법

9) 히스토그램 역투영

히스토그램 역투영

■ 히스토그램 역투영(Histogram backprojection)

- 영상의 각 픽셀이 주어진 히스토그램 모델에 얼마나 일치하는지를 검사하는 방법
- 임의의 색상 영역을 검출할 때 효과적



히스토그램 역투영

■ 히스토그램 역투영 함수

```
cv2.calcBackProject(images, channels, hist, ranges, scale, dst=None) -> dst
```

- images: 입력 영상 리스트
- channels: 역투영 계산에 사용할 채널 번호 리스트
- hist: 입력 히스토그램 (numpy.ndarray)
- ranges: 히스토그램 각 차원의 최솟값과 최댓값으로 구성된 리스트
- scale: 출력 역투영 행렬에 추가적으로 곱할 값
- dst: 출력 역투영 영상. 입력 영상과 동일 크기, cv2.CV_8U.

히스토그램 역투영

```
import sys
import numpy as np
import cv2

# 입력 영상에서 ROI를 지정하고, 히스토그램 계산

src = cv2.imread('cropland.png')

if src is None:
    print('Image load failed!')
    sys.exit()

x, y, w, h = cv2.selectROI(src)

src_ycrCb = cv2.cvtColor(src, cv2.COLOR_BGR2YCrCb)
crop = src_ycrCb[y:y+h, x:x+w]

channels = [1, 2]
cr_bins = 128
cb_bins = 128
histSize = [cr_bins, cb_bins]
cr_range = [0, 256]
cb_range = [0, 256]
ranges = cr_range + cb_range

hist = cv2.calcHist([crop], channels, None, histSize, ranges)
hist_norm = cv2.normalize(cv2.log(hist+1), None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)
```


히스토그램 역투영

입력 영상 전체에 대해 히스토그램 역투영

```
backproj = cv2.calcBackProject([src_ycrb], channels, hist, ranges, 1)  
dst = cv2.copyTo(src, backproj)
```

```
cv2.imshow('backproj', backproj)  
cv2.imshow('hist_norm', hist_norm)  
cv2.imshow('dst', dst)  
cv2.waitKey()  
cv2.destroyAllWindows()
```